

The Definitive Guide to Yii 2.0

Download ▾

English ▾

Version 2.0 ▾

SideNav

Search the Guide...

INTRODUCTION
GETTING STARTED
APPLICATION STRUCTURE
HANDLING REQUESTS
KEY CONCEPTS
WORKING WITH DATABASES
GETTING DATA FROM USERS
Creating Forms (/doc/guide/2.0/en/input-forms)
Validating Input (/doc/guide/2.0/en/input-validation)
Uploading Files (/doc/guide/2.0/en/input-file-upload)
Collecting Tabular Input (/doc/guide/2.0/en/input-tabular-input)
Getting Data for Multiple Models (/doc/guide/2.0/en/input-multiple-models)
Extending ActiveForm on the Client Side (/doc/guide/2.0/en/input-form-javascript)
DISPLAYING DATA
SECURITY
CACHING
RESTFUL WEB SERVICES
DEVELOPMENT TOOLS
TESTING
SPECIAL TOPICS
WIDGETS
HELPERS

☆ 4 followers

Uploading Files

1. [Creating Models](#)
2. [Rendering File Input](#)
3. [Wiring Up](#)
4. [Uploading Multiple Files](#)

Uploading files in Yii is usually done with the help of [yii\web\UploadedFile](#) ([/doc/api/2.0/yii-web-uploadedfile](#)), which encapsulates each uploaded file as an `UploadedFile` object. Combined with [yii\widgets\ActiveForm](#) ([/doc/api/2.0/yii-widgets-activeform](#)) and [models](#) ([/doc/guide/2.0/en/structure-models](#)), you can easily implement a secure file uploading mechanism.

Creating Models

Like working with plain text inputs, to upload a single file you would create a model class and use an attribute of the model to keep the uploaded file instance. You should also declare a validation rule to validate the file upload. For example,

```
namespace app\models;

use yii\base\Model;
use yii\web\UploadedFile;

class UploadForm extends Model
{
    /**
     * @var UploadedFile
     */
    public $imageFile;

    public function rules()
    {
        return [
            [['imageFile'], 'file', 'skipOnEmpty' => false, 'extensions' => 'png, jpg'],
        ];
    }

    public function upload()
    {
        if ($this->validate()) {
            $this->imageFile->saveAs('uploads/' . $this->imageFile->baseName . '.' . $this->imageFile->extension);
            return true;
        } else {
            return false;
        }
    }
}
```

In the code above, the `imageFile` attribute is used to keep the uploaded file instance. It is associated with a `file` validation rule which uses [yii\validators\FileValidator](#) ([/doc/api/2.0/yii-validators-filevalidator](#)) to ensure a file with extension name `png` or `jpg` is uploaded. The `upload()` method will perform the validation and save the uploaded file on the server.

The `file` validator allows you to check file extensions, size, MIME type, etc. Please refer to the [Core Validators](#) ([/doc/guide/2.0/en/tutorial-core-validators#file](#)) section for more details.

Tip: If you are uploading an image, you may consider using the `image` validator instead. The `image` validator is implemented via `yii\validators\ImageValidator` (</doc/api/2.0/yii-validators-imagevalidator>) which verifies if an attribute has received a valid image that can be then either saved or processed using the [Imagine Extension](#).

Rendering File Input

Next, create a file input in a view:

```
<?php
use yii\widgets\ActiveForm;
?>

<?php $form = ActiveForm::begin(['options' => ['enctype' => 'multipart/form-data']]) ?>

    <?= $form->field($model, 'imageFile')->fileInput() ?>

    <button>Submit</button>

<?php ActiveForm::end() ?>
```

It is important to remember that you add the `enctype` option to the form so that the file can be properly uploaded. The `fileInput()` call will render a `<input type="file">` tag which will allow users to select a file to upload.

Tip: since version 2.0.8, [fileInput](/doc/api/2.0/yii-widgets-activefield#fileInput()-detail) ([/doc/api/2.0/yii-widgets-activefield#fileInput\(\)-detail](/doc/api/2.0/yii-widgets-activefield#fileInput()-detail)) adds `enctype` option to the form automatically when file input field is used.

Wiring Up

Now in a controller action, write the code to wire up the model and the view to implement file uploading:

```
namespace app\controllers;

use Yii;
use yii\web\Controller;
use app\models\UploadForm;
use yii\web\UploadedFile;

class SiteController extends Controller
{
    public function actionUpload()
    {
        $model = new UploadForm();

        if (Yii::$app->request->isPost) {
            $model->imageFile = UploadedFile::getInstance($model, 'imageFile');
            if ($model->upload()) {
                // file is uploaded successfully
                return;
            }
        }

        return $this->render('upload', ['model' => $model]);
    }
}
```

In the above code, when the form is submitted, the `yii\web\UploadedFile::getInstance()` ([/doc/api/2.0/yii-web-uploadedfile#getinstance\(\)-detail](https://www.yiiframework.com/doc/api/2.0/yii-web-uploadedfile#getinstance()-detail)) method is called to represent the uploaded file as an `UploadedFile` instance. We then rely on the model validation to make sure the uploaded file is valid and save the file on the server.

Uploading Multiple Files

You can also upload multiple files at once, with some adjustments to the code listed in the previous subsections.

First you should adjust the model class by adding the `maxFiles` option in the `file` validation rule to limit the maximum number of files allowed to upload. Setting `maxFiles` to `0` means there is no limit on the number of files that can be uploaded simultaneously. The maximum number of files allowed to be uploaded simultaneously is also limited with PHP directive `max_file_uploads` [↗](#), which defaults to 20. The `upload()` method should also be updated to save the uploaded files one by one.

```

namespace app\models;

use yii\base\Model;
use yii\web\UploadedFile;

class UploadForm extends Model
{
    /**
     * @var UploadedFile[]
     */
    public $imageFiles;

    public function rules()
    {
        return [
            [['imageFiles'], 'file', 'skipOnEmpty' => false, 'extensions' => 'png, jpg', 'maxFiles' => 4],
        ];
    }

    public function upload()
    {
        if ($this->validate()) {
            foreach ($this->imageFiles as $file) {
                $file->saveAs('uploads/' . $file->baseName . '.' . $file->extension);
            }
            return true;
        } else {
            return false;
        }
    }
}

```

In the view file, you should add the `multiple` option to the `fileInput()` call so that the file upload field can receive multiple files. You also need to change `imageFiles` to `imageFiles[]` so that the attribute values are submitted as an array:

```

<?php
use yii\widgets\ActiveForm;
?>

<?php $form = ActiveForm::begin(['options' => ['enctype' => 'multipart/form-data']]) ?>

    <?= $form->field($model, 'imageFiles[]')->fileInput(['multiple' => true, 'accept' => 'image/*']) ?>

    <button>Submit</button>

<?php ActiveForm::end() ?>

```

And finally in the controller action, you should call `UploadedFile::getInstances()` instead of `UploadedFile::getInstance()` to assign an array of `UploadedFile` instances to `UploadForm::imageFiles`.

```

namespace app\controllers;

use Yii;
use yii\web\Controller;
use app\models\UploadForm;
use yii\web\UploadedFile;

class SiteController extends Controller
{
    public function actionUpload()
    {
        $model = new UploadForm();

        if (Yii::$app->request->isPost) {
            $model->imageFiles = UploadedFile::getInstances($model, 'imageFiles');
            if ($model->upload()) {
                // file is uploaded successfully
                return;
            }
        }

        return $this->render('upload', ['model' => $model]);
    }
}

```

Validating Input (/doc/guide/2.0/en/input-validation)

Go to Top

Collecting Tabular Input (/doc/guide/2.0/en/input-tabular-input)



Found a typo or you think this page needs improvement?

[Edit it on github](#) !

User Contributed Notes ¹

#21024

4 [./doc/guide/2.0/en/input-file-upload](#) 0
[./doc/guide/2.0/en/input-file-upload](#)

If your model rule for the file attribute only specifies the *extensions* options, this might often validate to *false*. The option *checkExtensionByMimeType* defaults to true.

Some files upload with mime type *text/plain*, like e.g. JSON files. The required MIME type would be *application/json*, so the validation by MIME type fails.

So set *checkExtensionByMimeType* to *false*, then the validation only goes via the filename extension.



jwerner (/user/147) at Mar 6, 2022, 7:04:51 PM

Leave a comment

[Signup](#) (/signup) or [Login](#) (/login) in order to comment.

 (<https://github.com/yiisoft>)  (<https://x.com/yiiframework>)  (<https://www.facebook.com/groups/yiitalk/>)  (</rss.xml>)

[Terms of service \(/tos\)](#)

[License \(/license\)](#)

[Website Source Code \(https://github.com/yiisoft-contrib/yiiframework.com\)](https://github.com/yiisoft-contrib/yiiframework.com)

© 2008 - 2025 Yii

Design: Eshill (<https://www.eshill.ru/>)

Supported by

Your donations



(<https://opencollective.com/yiisoft>)



(<https://www.jetbrains.com/?from=yii>)